16

分段

到目前为止，我们一直把每个进程的整个地址空间都放进内存。有了基址和界限寄存器，操作系统可以很容易地把进程重定位到物理内存的不同位置。不过，你可能已经注意到我们这些地址空间有个有意思的地方：在栈和堆之间，正中间有一大块“空闲”空间。

从图 16.1 可以想见，虽然栈和堆之间的空间并没有被进程使用，但当我们把整个地址空间重定位到物理内存中某处时，这块空间照样占着物理内存；因此，用一对基址和界限寄存器来虚拟化内存，这种简单的做法很浪费。而且，当整个地址空间放不进内存时，程序就很难运行；所以，基址加界限并不像我们希望的那样灵活。于是：

关键问题：如何支持大地址空间

栈和堆之间可能有大量空闲空间，这样的大地址空间该如何支持？注意，在我们的例子中，地址空间都很小（只是假想的），浪费看起来还不算严重。但想象一下 32 位的地址空间（4 GB 大小）：一个典型的程序只会用到几 MB 内存，却仍然要求整个地址空间都驻留在内存中。

16.1 分段：泛化的基址/界限

为了解决这个问题，人们提出了一个想法，叫作**分段**（segmentation）。这个想法相当古老，至少可以追溯到 20 世纪 60 年代初 [H61, G62]。思路很简单：与其在 MMU 中只放一对基址和界限寄存器，为什么不给地址空间的每个逻辑**段**（segment）都配一对呢？段就是地址空间中一段特定长度的连续区域。在我们一直使用的典型地址空间里，有三个逻辑上不同的段：代码、栈和堆。有了分段，操作系统就可以把这三个段分别放到物理内存的不同位置，从而避免未使用的虚拟地址空间占满物理内存。

我们来看一个例子。假设我们要把图 16.1 中的地址空间放进物理内存。每个段都有一对基址和界限寄存器，所以可以把每个段独立地放进物理内存。例如，图 16.2（第 3 页）展示了一个 64 KB 的物理内存，里面放着这三个段（另有 16 KB 留给操作系统）。

从图中可以看到，物理内存中只为用到的内存分配了空间。因此，即使地址空间很大、其中有大量未使用的部分（我们有时把这种地址空间称为**稀疏地址空间**，sparse address space），也能容纳下来。

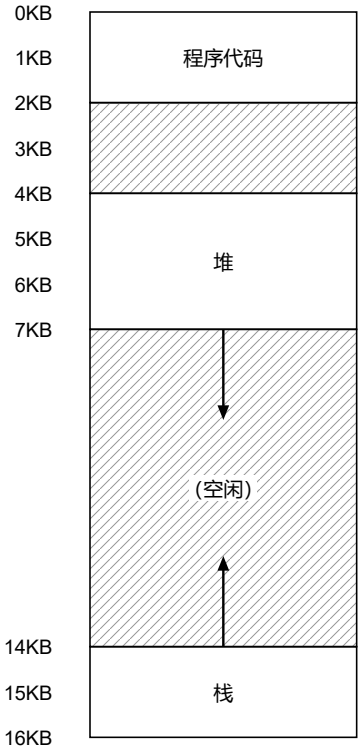


图 16.1：**地址空间（再看一次）**

MMU 中支持分段所需的硬件结构正如你所料：在这个例子里，就是三对基址和界限寄存器。下面的图 16.3 给出了上例中的寄存器值；每个界限寄存器保存的是一个段的大小。

从图中可以看到，代码段放在物理地址 32 KB 处，大小为 2 KB；堆段放在 34 KB 处，大小为 3 KB。这里的大小寄存器和前面介绍的界限寄存器完全一样：它告诉硬件这个段中到底有多少字节是有效的（硬件由此就能判断程序何时越界，访问了非法地址）。

我们用图 16.1 中的地址空间做一次地址转换。假设程序引用了虚拟地址 100，从图 16.1（第 2 页）可以直观地看到，它位于代码段中。引用发生时（比如取指令时），硬件会把基址加上这个段内的*偏移量*（这里是 100），得到所需的物理地址：100 + 32 KB，即 32868。然后它会检查这个地址是否在界限之内（100 小于 2 KB），确认在界限内之后，就向物理内存地址 32868 发出引用。

现在来看堆中的一个地址：虚拟地址 4200（同样参见图 16.1）。如果我们直接把虚拟地址 4200 加到堆的基址（34 KB）上，得到的物理地址是 39016，这*并不*是正确的物理地址。我们首先要做的，是提取出堆内的*偏移量*，也就是这个地址引用的是这个*段*中的哪个（或哪些）字节。因为堆从虚拟地址 4 KB（4096）开始，所以 4200 的偏移量实际上是 4200 减去 4096，即 104。然后我们把这个偏移量（104）加到基址寄存器中的物理地址（34K）上，得到想要的结果：34920。

 © 2008–25, ARPACI-DUSSEAU

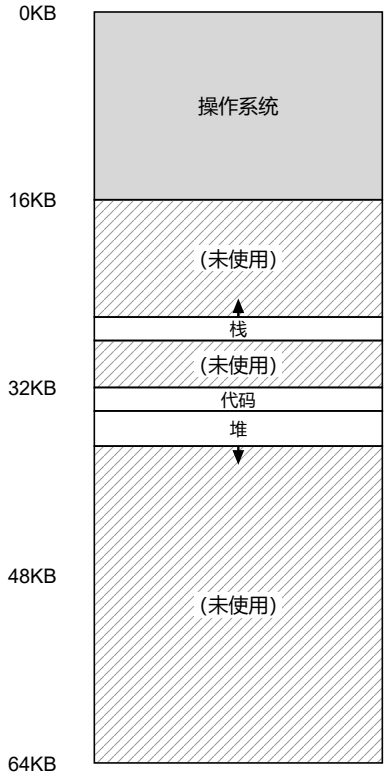


图 16.2：**把段放进物理内存**

如果我们试图引用一个非法地址（即 7 KB 或更大的虚拟地址），也就是超出堆末尾的地址，会怎样？你可以想象会发生什么：硬件检测到地址越界，陷入操作系统，很可能导致

段	基址	大小
代码	32K	2K
堆	34K	3K
栈	28K	2K

图 16.3：**段寄存器的值**

补充：段错误

段错误（segmentation fault）也叫段违规（segmentation violation），这个术语源自在分段机器上对非法地址的内存访问。好笑的是，这个术语一直沿用至今，就连完全不支持分段的机器上也在用。不过，要是你搞不清自己的代码为什么老是出错，那就不那么好笑了。

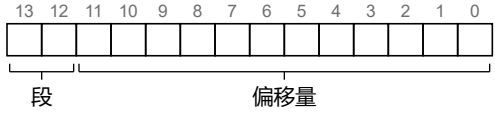
 © 2008–25, ARPACI-DUSSEAU

这个出错的进程被终止。现在你知道这个让所有 C 程序员都闻之色变的著名术语是怎么来的了：**段违规**或**段错误**。

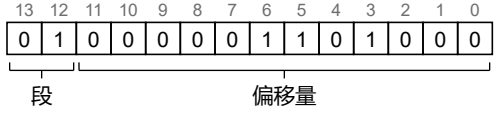
16.2 引用的是哪个段?

硬件在转换时会用到段寄存器。它怎么知道某个地址在段内的偏移量，以及这个地址引用的是哪个段呢?

一种常见的方法有时称为**显式**（explicit）方法：根据虚拟地址的最高几位，把地址空间划分成段；VAX/VMS 系统就使用了这种技术 [LL82]。在上面的例子中，我们有三个段，因此需要两位来完成这项任务。如果用 14 位虚拟地址的最高两位来选择段，虚拟地址就是这个样子：



在我们的例子中，如果最高两位是 00，硬件就知道这个虚拟地址在代码段中，于是用代码段的基址和界限把地址重定位到正确的物理位置。如果最高两位是 01，硬件就知道地址在堆中，于是使用堆的基址和界限。我们把上面那个堆中的虚拟地址（4200）拿来转换一下，确保你理解了这一点。虚拟地址 4200 的二进制形式如下：



从图中可以看到，最高两位（01）告诉硬件我们引用的是哪个段。低 12 位是段内的偏移量：0000 0110 1000，即十六进制的 0x068，或十进制的 104。因此，硬件只需取前两位来决定使用哪个段寄存器，再把后面 12 位当作段内偏移量。把基址寄存器加上偏移量，硬件就得到了最终的物理地址。注意，有了偏移量，界限检查也变得简单：只需检查偏移量是否小于界限；如果不小于，这个地址就是非法的。因此，如果基址和界限是数组（每个段一项），硬件为了得到所需的物理地址，所做的事情大致如下：

```
1 // 取 14 位虚拟地址的最高 2 位
2 Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3 // 再取偏移量
4 Offset  = VirtualAddress & OFFSET_MASK
5 if (Offset >= Bounds[Segment])
6     RaiseException(PROTECTION_FAULT)
7 else
8     PhysAddr = Base[Segment] + Offset
9     Register = AccessMemory(PhysAddr)
```

在我们这个贯穿始终的例子中，可以把上面的常量填上具体的值：`SEG_MASK` 设为 `0x3000`，`SEG_SHIFT` 设为 `12`，`OFFSET_MASK` 设为 `0xFFF`。

你可能还注意到，用最高两位选段，而我们只有三个段（代码、堆、栈），这样地址空间中就有一个段闲置了。为了充分利用虚拟地址空间（避免出现闲置的段），有些系统把代码和堆放在同一个段里，这样只需一位就能选出用哪个段 [LL82]。

用最高的若干位来选段，还有一个问题：它限制了虚拟地址空间的使用。具体来说，每个段的大小都有上限，在我们的例子中是 4 KB（用最高两位选段，意味着 16 KB 的地址空间被切成四块，在这个例子里每块 4 KB）。如果正在运行的程序想把某个段（比如堆或栈）扩大到超过这个上限，那就没办法了。

硬件还有其他办法来判断某个地址属于哪个段。在**隐式**（implicit）方法中，硬件根据地址是怎样产生的来确定段。比如，如果地址是由程序计数器产生的（也就是取指令），那么这个地址就在代码段中；如果地址基于栈指针或基址指针，那它一定在栈段中；其他地址则都在堆中。

16.3 栈怎么办?

到目前为止，我们漏掉了地址空间中一个重要的组成部分：栈。在上面的图中，栈被重定位到了物理地址 28 KB，但它有一个关键的不同：*它是反向增长的*（即朝着低地址方向增长）。在物理内存中，它从 28 KB[1] 处“开始”，往回增长到 26 KB，对应虚拟地址 16 KB 到 14 KB；它的地址转换必须换一种方式进行。

首先需要一点额外的硬件支持。除了基址和界限值，硬件还需要知道段朝哪个方向增长（比如用一个位来表示：段朝正方向增长时设为 1，朝负方向增长时设为 0）。硬件需要记录的信息更新后如图 16.4 所示：

硬件知道段可以朝负方向增长之后，转换这类虚拟地址的方式也要稍作改变。我们拿一个栈中的虚拟地址为例，看看它是怎么转换的。

在这个例子中，假设我们要访问虚拟地址 15 KB，它应该映射到物理地址 27 KB。这个虚拟地址的二进制形式是 11 1100 0000 0000（十六进制 0x3C00）。硬件用最高两位（11）指定段，剩下的偏移量是 3 KB。为了得到正确的负偏移量，我们必须用 3 KB 减去段的最大大小：在这个例子里，段最大可以是 4 KB，因此正确的负偏移量是 3 KB 减 4 KB，即 −1 KB。只要把这个负偏移量（−1 KB）加到基址（28 KB）上，就得到了正确的物理地址：27 KB。至于界限检查，只要确保负偏移量的绝对值小于或等于段的当前大小（这里是 2 KB）即可。

段	基址	大小（最大 4K）	正向增长?
代码$_{00}$	32K	2K	1
堆$_{01}$	34K	3K	1
栈$_{11}$	28K	2K	0

图 16.4：**段寄存器（支持负向增长）**

[1]为简单起见，我们说栈从 28 KB 处“开始”，但这个值其实是紧挨在反向增长区域下方的那个字节；第一个有效字节实际上是 28 KB 减 1。相比之下，正向增长的区域从段的第一个字节的地址开始。我们这样处理，是因为这样计算物理地址最直接：物理地址就是基址加上负偏移量。

段	基址	大小（最大 4K）	正向增长?	保护
代码$_{00}$	32K	2K	1	读-执行
堆$_{01}$	34K	3K	1	读-写
栈$_{11}$	28K	2K	0	读-写

图 16.5：**段寄存器的值（带保护位）**

16.4 支持共享

随着对分段的支持日益完善，系统设计者很快意识到，只要再加一点硬件支持，就能获得新的效率提升。具体来说，为了节省内存，有时在地址空间之间**共享**（share）某些内存段会很有用。尤其是**代码共享**（code sharing），它很常见，如今的系统里也还在用。

为了支持共享，我们需要硬件再提供一点支持，也就是**保护位**（protection bits）。基本的做法是给每个段加几个位，表明程序能否读写这个段，也许还表明能否执行段内的代码。把代码段设为只读后，同一份代码就可以在多个进程之间共享，而不用担心破坏隔离性；每个进程仍然以为自己在访问私有的内存，而操作系统在暗中共享着这块进程无法修改的内存，于是假象得以维持。

图 16.5 举例展示了硬件（和操作系统）额外记录的信息。可以看到，代码段被设为可读、可执行，因此内存中同一个物理段可以映射到多个虚拟地址空间中。

有了保护位，前面描述的硬件算法也得跟着改。除了检查虚拟地址是否在界限之内，硬件还必须检查某次访问是否被允许。如果用户进程试图写只读段，或者从不可执行的段中执行代码，硬件就应该触发异常，让操作系统来处理这个出错的进程。

16.5 细粒度与粗粒度的分段

到目前为止，我们的例子大多针对只有几个段（即代码、栈、堆）的系统；这种分段可以看作**粗粒度**（coarse-grained）的，因为它把地址空间切成相对较大、较粗的几块。不过，一些早期系统（例如 Multics [CV65, DD68]）更加灵活，允许地址空间由大量较小的段组成，这称为**细粒度**（fine-grained）分段。

要支持很多段，还需要更多的硬件支持，即在内存中存放某种**段表**（segment table）。这类段表通常支持创建非常多的段，从而让系统能比我们前面讨论的更灵活地使用段。例如，像 Burroughs B5000 这样的早期机器支持成千上万个段，并期望编译器把代码和数据切分成独立的段，再由操作系统和硬件来支持 [RK68]。当时的想法是，有了细粒度的段，操作系统就能更清楚地了解哪些段在使用、哪些没有，从而更有效地利用主存。

16.6 操作系统支持

现在你应该对分段的工作原理有了基本的了解。系统运行时，地址空间的各个部分被重定位到物理内存中；与整个地址空间只用一对基址/界限寄存器的简单方法相比，这样能节省大量物理内存。具体来说，栈和堆之间所有未使用的空间都不必在物理内存中分配，于是我们能在物理内存中放下更多地址空间，并为每个进程支持大而稀疏的虚拟地址空间。

 © 2008–25, ARPACI-DUSSEAU

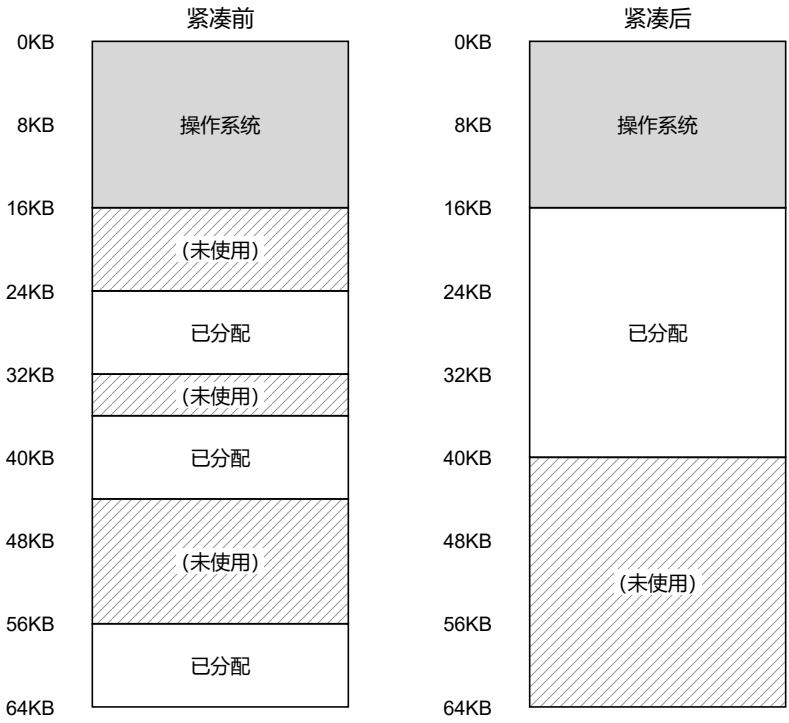


图 16.6：**紧凑前与紧凑后的内存**

然而，分段给操作系统带来了一些新问题。第一个是老问题了：上下文切换时操作系统该做什么？现在你应该能猜到：必须保存和恢复段寄存器。显然，每个进程都有自己的虚拟地址空间，操作系统必须在让进程重新运行之前，把这些寄存器设置正确。

第二个问题是段增长（或者缩小）时操作系统如何配合。比如，程序可能调用 `malloc()` 来分配一个对象。有时，现有的堆就能满足这个请求，于是 `malloc()` 会为这个对象找到空闲空间，并把指向它的指针返回给调用者。但有时，堆段本身需要增长。这时，内存分配库会发起系统调用来扩大堆（例如传统 UNIX 中的 `sbrk()` 系统调用）。然后，操作系统（通常）会提供更多空间，把段大小寄存器更新为新的（更大的）大小，并通知分配库操作成功；分配库就可以为新对象分配空间，并成功返回给调用它的程序。注意，如果已经没有更多物理内存可用，或者操作系统认为调用进程占用的内存已经太多了，操作系统也可能拒绝这个请求。

最后一个、也许是最重要的问题，是管理物理内存中的空闲空间。创建新的地址空间时，操作系统必须能在物理内存中为它的各个段找到空间。之前我们假设每个地址空间大小都相同，因此可以把物理内存看作一组槽位，每个进程放进其中一个。现在，每个进程有好几个段，而且每个段的大小可能都不一样。

 © 2008–25, ARPACI-DUSSEAU

提示：如果有 1000 种解决方案，那就没有一种真正好的
已经有这么多不同的算法试图把外部碎片降到最低，这说明了一个更深层的事实：这个问题不存在一种“最好”的解法。因此，我们只能退而求其次，选一个合理的方案，并希望它足够好。唯一真正的解决办法（我们在接下来的章节中会看到）是彻底绕开这个问题：永远不以可变大小的块来分配内存。

由此产生的普遍问题是：物理内存很快就会布满一个个小的空闲空间“空洞”，导致很难为新的段分配空间，也很难扩大已有的段。我们把这个问题称为**外部碎片**（external fragmentation）[R69]；参见图 16.6（左）。

在这个例子中，来了一个进程，想分配一个 20 KB 的段。例子中一共有 24 KB 空闲，但它们并不是一整块连续的空间（而是三块不连续的区域）。因此，操作系统无法满足这个 20 KB 的请求。请求扩大某个段时也可能出现类似的问题：如果段后面紧接着的那些物理空间不可用，即使物理内存的其他地方还有空闲字节，操作系统也只能拒绝这个请求。

解决这个问题的一种方法是重新排列现有的段，把物理内存**紧凑**（compact）起来。例如，操作系统可以停下正在运行的进程，把它们的数据复制到一块连续的内存区域中，修改它们的段寄存器值，让它们指向新的物理位置，从而腾出一大块空闲内存。这样一来，新的分配请求就能成功了。不过，紧凑的代价很高，因为复制段是内存密集型操作，通常要占用相当多的处理器时间；紧凑后的物理内存见图 16.6（右）。讽刺的是，紧凑还会让扩大已有段的请求更难满足，可能因此需要进一步重新排列。

一种更简单的办法也许是使用空闲链表管理算法，尽量保留大块可供分配的内存。人们用过的方法真有好几百种，包括经典算法，如**最佳适应**（best-fit，维护一个空闲空间链表，把能满足请求且大小最接近的那块返回给请求者）、**最坏适应**（worst-fit）、**首次适应**（first-fit），以及更复杂的方案，如**伙伴算法**（buddy algorithm）[K68]。如果你想进一步了解这类算法，Wilson 等人写的一篇出色的综述是个很好的起点 [W+95]；你也可以等到后面的章节，我们会介绍其中一些基础内容。不过遗憾的是，无论算法多聪明，外部碎片依然存在；好的算法只是尽量减少它。

16.7 小结

分段解决了不少问题，帮助我们实现了更有效的内存虚拟化。分段不只是实现了动态重定位，还能更好地支持稀疏地址空间，因为它避免了地址空间各逻辑段之间可能存在的大量内存浪费。它也很快，因为分段所需的运算很简单，非常适合用硬件实现；地址转换的开销极小。此外还有一个附带的好处：代码共享。如果把代码放在单独的段中，这个段就有可能被多个正在运行的程序共享。

然而，我们也看到了，在内存中分配大小可变的段会带来一些我们希望克服的问题。第一个问题就是前面讨论过的外部碎片。由于段的大小可变，空闲内存会被切成大小不一的碎块，因此满足内存分配请求可能很困难。我们可以尝试使用聪明的算法 [W+95]，或者定期紧凑内存，但这个问题是根本性的，很难避免。

第二个、也许更重要的问题是，分段还是不够灵活，无法支持我们完全通用的稀疏地址空间。例如，如果一个很大、但只有零星部分被用到的堆全在一个逻辑段里，那么整个堆仍然必须驻留在内存中才能被访问。换句话说，如果地址空间的使用方式与底层分段的设计初衷不完全吻合，分段的效果就不太好。因此，我们需要找到一些新的解决方案。准备好了吗?

 © 2008–25, ARPACI-DUSSEAU

参考文献

[CV65] "Introduction and Overview of the Multics System" by F. J. Corbato, V. A. Vyssotsky. Fall Joint Computer Conference, 1965. *One of five papers presented on Multics at the Fall Joint Computer Conference; oh to be a fly on the wall in that room that day!*

[DD68] "Virtual Memory, Processes, and Sharing in Multics" by Robert C. Daley and Jack B. Dennis. Communications of the ACM, Volume 11:5, May 1968. *An early paper on how to perform dynamic linking in Multics, which was way ahead of its time. Dynamic linking finally found its way back into systems about 20 years later, as the large X-windows libraries demanded it. Some say that these large X11 libraries were MIT's revenge for removing support for dynamic linking in early versions of UNIX!*

[G62] "Fact Segmentation" by M. N. Greenfield. Proceedings of the SJCC, Volume 21, May 1962. *Another early paper on segmentation; so early that it has no references to other work.*

[H61] "Program Organization and Record Keeping for Dynamic Storage" by A. W. Holt. Communications of the ACM, Volume 4:10, October 1961. *An incredibly early and difficult to read paper about segmentation and some of its uses.*

[I09] "Intel 64 and IA-32 Architectures Software Developer's Manuals" by Intel. 2009. Available: http://www.intel.com/products/processor/manuals. *Try reading about segmentation in here (Chapter 3 in Volume 3a); it'll hurt your head, at least a little bit.*

[K68] "The Art of Computer Programming: Volume I" by Donald Knuth. Addison-Wesley, 1968. *Knuth is famous not only for his early books on the Art of Computer Programming but for his typesetting system TeX which is still a powerhouse typesetting tool used by professionals today, and indeed to typeset this very book. His tomes on algorithms are a great early reference to many of the algorithms that underlie computing systems today.*

[L83] "Hints for Computer Systems Design" by Butler Lampson. ACM Operating Systems Review, 15:5, October 1983. *A treasure-trove of sage advice on how to build systems. Hard to read in one sitting; take it in a little at a time, like a fine wine, or a reference manual.*

[LL82] "Virtual Memory Management in the VAX/VMS Operating System" by Henry M. Levy, Peter H. Lipman. IEEE Computer, Volume 15:3, March 1982. *A classic memory management system, with lots of common sense in its design. We'll study it in more detail in a later chapter.*

[RK68] "Dynamic Storage Allocation Systems" by B. Randell and C.J. Kuehner. Communications of the ACM, Volume 11:5, May 1968. *A nice overview of the differences between paging and segmentation, with some historical discussion of various machines.*

[R69] "A note on storage fragmentation and program segmentation" by Brian Randell. Communications of the ACM, Volume 12:7, July 1969. *One of the earliest papers to discuss fragmentation.*

[W+95] "Dynamic Storage Allocation: A Survey and Critical Review" by Paul R. Wilson, Mark S. Johnstone, Michael Neely, David Boles. International Workshop on Memory Management, Scotland, UK, September 1995. *A great survey paper on memory allocators.*

 © 2008–25, ARPACI-DUSSEAU

作业（模拟）

这个程序可以让你看到，在分段系统中地址转换是如何进行的。详情请见 README。

问题

1. 首先，我们用一个很小的地址空间来转换几个地址。下面是一组简单的参数，配上几个不同的随机种子；你能转换这些地址吗？
   - `segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 0`
   - `segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 1`
   - `segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 2`
2. 现在，看看我们是否理解了自己构造的这个小地址空间（使用上一题的参数）。段 0 中最高的合法虚拟地址是多少？段 1 中最低的合法虚拟地址呢？在整个地址空间中，最低和最高的非法地址分别是多少？最后，你会怎样用 `-A` 选项运行 `segmentation.py`，来检验自己的答案对不对？
3. 假设在 128 字节的物理内存中有一个 16 字节的小地址空间。对于指定的地址流，你要怎样设置基址和界限，才能让模拟器产生下面的转换结果：有效、有效、违规……违规、有效、有效？假设参数如下：

```
segmentation.py -a 16 -p 128
    -A 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15
    --b0 ? --l0 ? --b1 ? --l1 ?
```

4. 假设我们想生成这样一个问题：随机生成的虚拟地址中大约 90% 是有效的（不是段违规）。应该如何配置模拟器？哪些参数对得到这个结果比较重要？
5. 你能让模拟器运行后没有一个虚拟地址是有效的吗？怎么做？

 © 2008–25, ARPACI-DUSSEAU